Advanced Programming for Scientists

Lecture 3

Virtualization
for
Reproducible Scientific Computing

v1.0

Prof. Mateusz Bawaj

A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

Why Virtualization in Science?

Problem:

Scientific software often fails not because of physics, but because of environment.

- Different OS versions
- Incompatible libraries
- GPU drivers mismatch
- “It works on my machine” syndrome
- Non-reproducible computational experiments

Why Virtualization in Science?

In computational physics and ML:

- Numerical reproducibility depends on compiler, BLAS, CUDA, kernel
- Environment drift corrupts long-term projects
- HPC clusters differ from local laptops

Virtualization = controlled computational environment

“Basic Linear Algebra Subprograms (BLAS) is a specification that prescribes a set of low-level routines for performing common linear algebra operations such as vector addition, scalar multiplication, dot products, linear combinations, and matrix multiplication. Although the BLAS specification is general, BLAS implementations are often optimized for speed on a particular machine...”

Reproducibility as a Scientific Requirement

Reproducibility is part of the scientific method.

In computational research this means:

- Same source code (git)
- Same dependencies (conda)
- Same compiler/interpreter (virtualization)
- Same runtime environment (virtualization)

Virtualization enables:

- Reproducible simulations
- Shareable ML training environments
- Long-term preservation of scientific workflows

Types of Virtualization

Two main approaches:

1) Full Virtualization (Virtual Machines)
   - Emulates entire hardware stack
   - Includes full OS
   - Example: Oracle VM VirtualBox
2) Container-based Virtualization
   - Shares host kernel
   - Isolates user-space
   - Lightweight
   - Example: Docker, Singularity (Apptainer)

Full Virtualization (Virtual Machines)

Architecture:

Host OS

→ Hypervisor

→ Guest OS

→ Applications

Advantages

- Strong isolation
- Different OS (e.g. Windows inside Linux)
- Useful for system-level experiments

Full Virtualization (Virtual Machines)

Disadvantages

- Heavy
- High memory usage
- Slower boot

Scientific use case

- Testing numerical code on different OS
- Reproducing legacy scientific software
- Teaching controlled environments

Containers (Light Virtualization)

Architecture:

Host OS

→ Container engine

→ Isolated user-space environments

No separate kernel.

Containers (Light Virtualization)

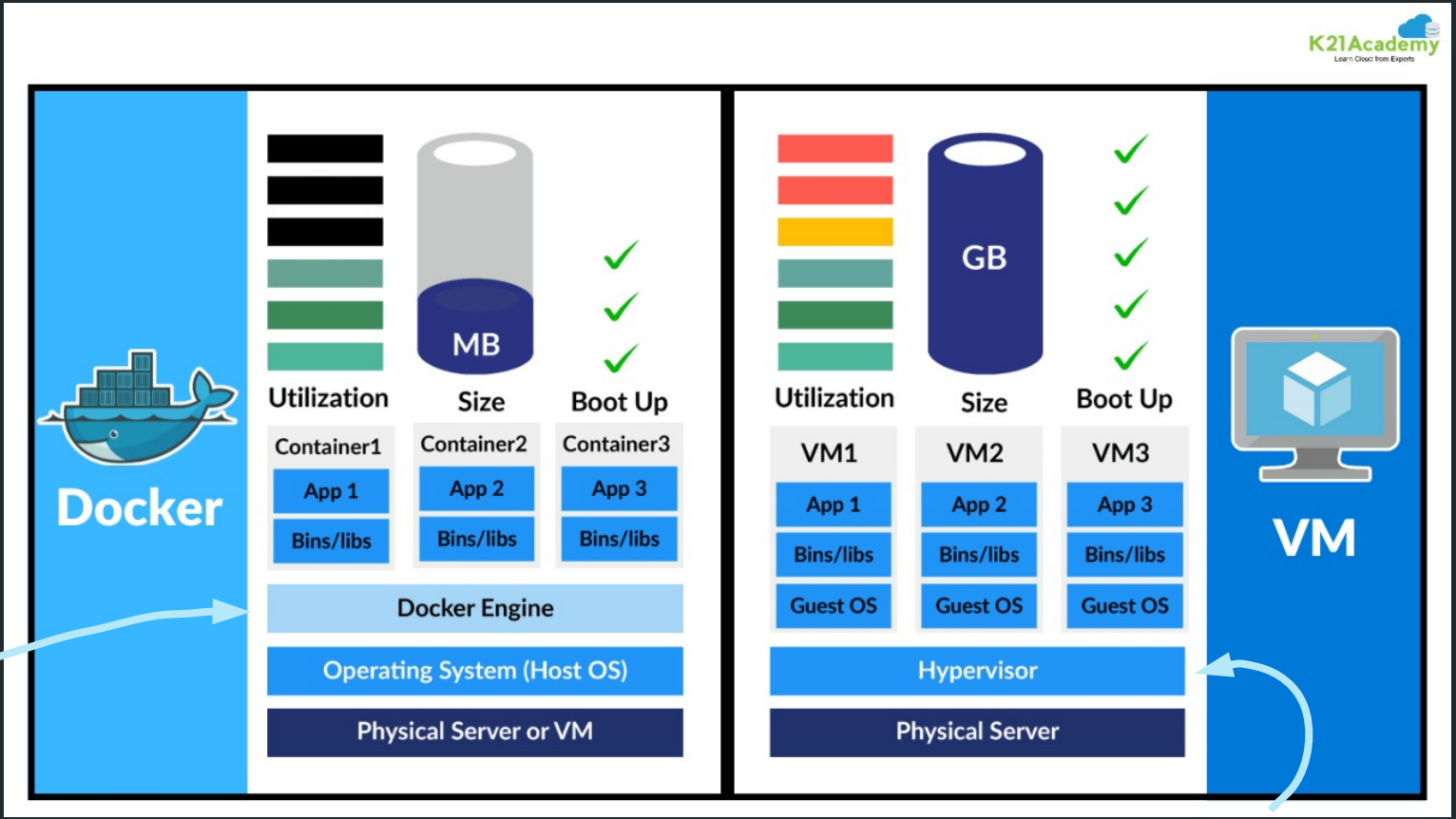
Advantages

- Lightweight
- Fast startup
- Ideal for CI/CD pipelines
- Perfect for scientific workflows

Disadvantages

- Same kernel as host
- Slightly weaker isolation

Virtualization comparison



does not virtualize hardware*

requires hardware extension for full virtualization

Why Containers Matter for Scientists

In computational physics / ML:

- Exact Python version matters
- Exact NumPy/BLAS build matters
- CUDA compatibility matters
- Compiler version matters

Containerization ensures identical environment across: Laptop, University server, HPC cluster, Cloud

Why Containers Matter for Scientists

Example:

- Time-domain simulator benchmarking
- Reproducible numerical integration tests
- ML training of cavity lock controller
- ...your examples?

Docker Basics

Using Docker

Key concepts:

- **Image** → blueprint
- **Container** → running instance
- `Dockerfile` → recipe
- **Registry** → image storage (Docker Hub)

Docker Basics

prepare **Dockerfile**

Example **Dockerfile** (scientific Python):

```
# ---- Base image ----
FROM python:3.11-slim

# ---- Install scientific packages ----
RUN pip install --no-cache-dir numpy scipy matplotlib

# ---- Create working directory ----
WORKDIR /app

# ---- Copy script into container ----
COPY simulate.py .

# ---- Create mount point for persistent data ----
RUN mkdir /data

# ---- Default command ----
CMD ["python", "simulate.py"]
```

Key points:

FROM pulls official Python base image

RUN installs scientific stack

COPY moves script inside container

/data is external persistent storage

simulate.py must be in the folder with the Dockerfile

Docker Basics

build the Image

From project directory:

```
$ docker build -t scientific-sim .
```

Git best practice:

- Commit Dockerfile
- Commit simulate.py
- Tag version when environment stabilizes

Docker Basics

run with mounted folder

```
$ docker run --rm \
  -v $(pwd)/data:/data \
  scientific-sim
```

Explanation:

- `-v host_folder:/data` mounts persistent storage
- Container writes results into host `data/`
- `--rm` removes container after execution

Now `data` folder contains results.

Git + Docker = (Scientific Reproducibility)^2

We integrate this lesson with **Versioning (Lesson 2)**.

Best practice:

Repository contains:

- Dockerfile
- requirements.txt
- README.md (with section about rebuilding the environment)

Git tracks:

- Source code
- Dockerfile
- Environment definition

Result: Anyone can clone and build identical environment

Git Workflow with Containers

Scientific workflow:

1) Create feature branch
2) Modify code
3) Update Dockerfile if dependencies change
4) Commit both code and environment
5) Push to remote repository
6) CI builds container automatically

This links to:

- Continuous Integration
- Automated testing inside container
- Reproducible pipelines

Containers in CI/CD

Typical scientific CI pipeline:

Push to GitHub

→ Build Docker image

→ Run unit tests

→ Run numerical methods tests

→ Compare outputs with reference data

This assures:

- Controlled umerical drift
- Coherent dependency updates
- Consistent simulation results

Containers on HPC

Modern HPC systems:

- Support container runtimes
- Use alternatives like Singularity (Apptainer)

Why?

- Security concerns
- Multi-user systems

Scientific example:

- Deploy same ML model
- Run Monte Carlo jobs
- Submit through HTCondor (future lesson)

When to Use Virtual Machines?

Use VM if:

- You need different OS kernel
- You test system drivers
- You simulate network topologies
- You test low-level hardware interactions

Example:

- Testing DAQ stack in controlled environment
- Trying different Linux distributions for real-time kernel

Performance Considerations

Containers:

- Near-native performance
- Small overhead

VM:

- More overhead
- Hardware virtualization layer

Speed considerations are important for:

- HPC simulations
- GPU acceleration

Always benchmark, never assume.

Hands-on Activity (In-class)

Exercise plan:

1) Create simple numerical script (ODE integration)
2) Track it with git
3) Create Dockerfile
4) Build container
5) Run simulation inside container
6) Compare results

Goal:

Understand that environment is part of scientific code.

Homework

1) Containerize your previous project
2) Add `Dockerfile` to repository
3) Commit and push
4) Tag version v0.1-containerized
5) Write short README section:
   - How to build
   - How to run

Key takeaways

Virtualization is not DevOps fashion.

It is:

- A scientific reproducibility tool
- A collaboration enabler
- A protection against environment decay
- A bridge between laptop and HPC

Git tracks code.

Containers track environment.

Science requires both.